

Evaluating and Improving the Performance and User Experience of Progressive Web Apps (PWAs) to Compete with Native Apps in Low-Connectivity Environments

Banan Almaqrami

July 24, 2026

1 Introduction

With the rapid evolution of web technology, traditional web applications face major challenges such as total reliance on internet connectivity and inferior performance compared to native apps. Progressive Web Apps (PWAs) have emerged as an innovative solution combining the flexibility of the web with the power of native apps through technologies like Service Workers and Caching, enabling fast and reliable user experiences even during network disruptions.

2 Research Problem

The research problem addresses the extent to which PWAs can provide a user experience that equals or closely approaches native applications, especially when handling big data or operating in environments with weak internet connectivity. These applications often face technical challenges related to cache memory management, background synchronization, and compatibility across various devices and browsers.

3 Objectives and Significance

3.1 Research Objectives:

- Analyze the technical architecture of PWAs and compare them with native applications.
- Develop a practical prototype of a PWA that supports offline mode.
- Measure and improve key performance metrics such as load time and response time using standard measurement tools.
- Evaluate user experience (UX) and user satisfaction regarding performance compared to traditional applications.

3.2 Research Significance:

- Reducing development costs by building a single cross-platform version instead of separate native apps for each operating system.
- Enhancing accessibility and response speed for users in regions with limited internet connectivity.

4 Research Questions

1. To what extent can PWAs simulate native app performance during internet outages?
2. What are the most efficient caching strategies to minimize load time?
3. How do push notifications and background execution features affect user engagement with the application?

5 Methodology and Proposed Tools

The research will follow an **Experimental and Developmental Approach**, including:

- **Development Environment:** Using modern frontend frameworks (such as React.js or Vue.js) to build the proposed application.
- **Core PWA Technologies:** Activating the Web App Manifest and integrating Service Workers to handle network requests and support offline mode.
- **Measurement and Testing Tools:** Using **Lighthouse** to measure performance and accessibility, alongside UX testing.

6 Literature Review

- Reviewing recent research comparing the performance of web apps and native apps.
- Focusing on literature discussing the efficiency of Service Workers and advanced caching mechanisms (Cache API / IndexedDB).
- **Research Gap:** Measuring performance in scenarios with very weak or frequently interrupted internet connections.

7 Timeline

- **Month 1:** Planning, analysis, and literature review.
- **Month 2:** Development, prototype building, and Service Workers integration.
- **Month 3:** Testing and measurement using Lighthouse and UX evaluation.
- **Month 4:** Data analysis, drafting the final report, and documenting results.

8 Expected Outcomes

- Delivering a PWA prototype that achieves high performance matching native apps in response speed.
- Significant reduction in recurrent load times through effective caching strategies.
- Providing a reference guide for developers on efficiently employing PWAs in limited-internet environments.

9 Preliminary References

References

- [1] MDN Web Docs, *Progressive Web Apps (PWAs) Documentation*, Mozilla Foundation.
- [2] W3C, *Progressive Web App Specifications and Standards*.
- [3] Google Developers, *PWA Checklist and Best Practices*.